MEMORIA PEC1 – HERRAMIENTAS HTML Y CSS MÁSTER UNIVERSITARIO EN DESARROLLO WEB – UOC

Diseño e implementación de un sitio web estático sobre gastronomía mediterránea

NOMBRE: IGNACIO YEBRA CABRERA
ASIGNATURA: HERRAMIENTAS HTML Y CSS
ACTIVIDAD: PEC1
FECHA: 02/04/2026

1. Introducción

El presente proyecto consiste en el desarrollo de un sitio web estático centrado en la gastronomía mediterránea, con especial atención a platos representativos de la tradición valenciana. El objetivo principal ha sido aplicar de forma integrada los conocimientos adquiridos en HTML y CSS, complementándolos con el uso de herramientas modernas de desarrollo como Parcel.

Más allá de la implementación funcional, se ha puesto el foco en aspectos que resultan fundamentales en el desarrollo web actual:

- uso correcto de HTML semántico
- accesibilidad del contenido
- organización y mantenibilidad del código
- coherencia estructural entre páginas
- separación clara entre contenido, presentación y comportamiento

El sitio se compone de cinco páginas principales:

- página de inicio
- página de categoría
- dos páginas de detalle
- página de enlaces y atribuciones

Todas ellas comparten una base estructural común, lo que permite una navegación consistente y una experiencia de usuario homogénea.

2. Proceso de creación de la plantilla con Parcel

El desarrollo se ha llevado a cabo utilizando Parcel como herramienta de construcción. La elección de Parcel responde a su bajo nivel de configuración inicial y a su capacidad para gestionar automáticamente recursos del proyecto.

El proceso seguido ha sido:

1. inicialización del proyecto mediante npm
2. instalación de Parcel como dependencia de desarrollo
3. definición de scripts en package.json para ejecutar el entorno
4. creación de una estructura de proyecto basada en la carpeta src

Organización del proyecto

Se ha adoptado una estructura clara y modular:

- src/ → contiene los documentos HTML
- assets/css/ → hojas de estilo
- assets/js/ → scripts

- assets/img/ → recursos gráficos

Esta organización permite separar responsabilidades y facilita la escalabilidad del proyecto.

Gestión de rutas

Uno de los aspectos más relevantes ha sido la gestión de rutas relativas. Durante el desarrollo se han utilizado rutas explícitas compatibles con Parcel, evitando el uso de rutas absolutas.

Por ejemplo:

```
<a href="../det1/index.html">Ver detalle</a>
```

Esta decisión se justifica porque:

- Parcel no resuelve correctamente rutas de carpeta (../) sin archivo explícito
- las rutas absolutas (/det1) pueden fallar en entornos de despliegue

Se trata de una decisión técnica que prioriza la robustez del proyecto frente a la comodidad inicial.

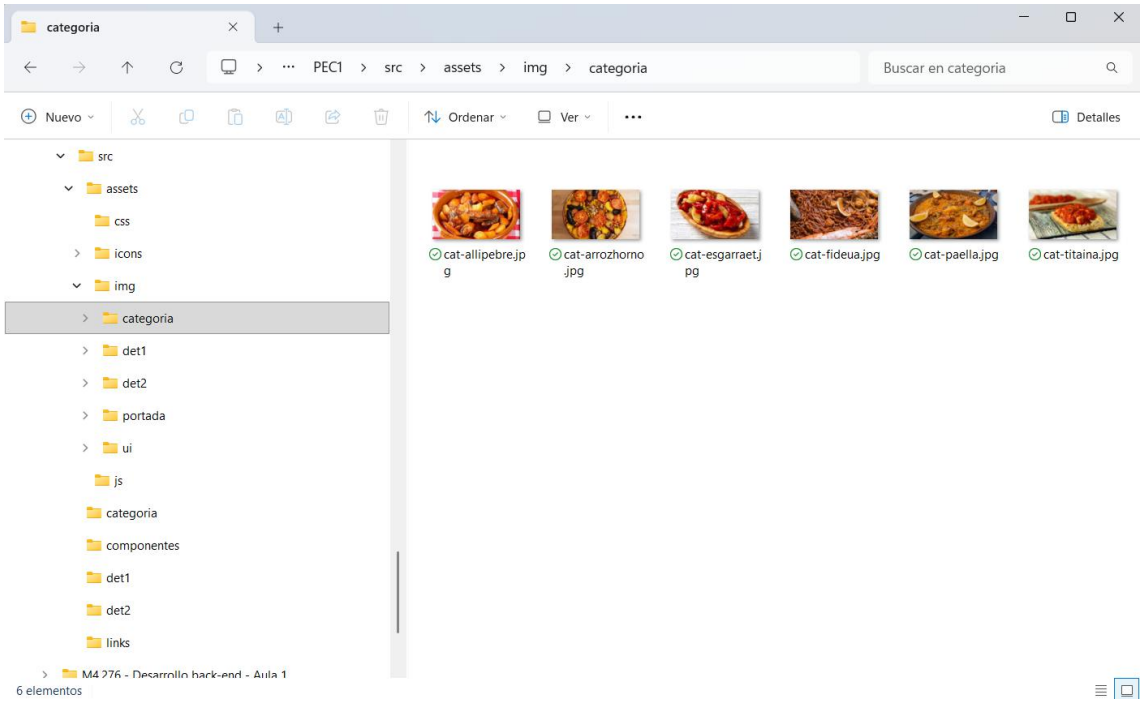

Figura 1. Estructura del proyecto. *Organización de carpetas y archivos utilizada para separar contenido, estilos, scripts y recursos gráficos*

Tras definir la estructura del proyecto, se procedió a la puesta en marcha del entorno de desarrollo mediante Parcel. Esta herramienta permite automatizar el proceso de compilación y servir el proyecto en un entorno local, facilitando la detección temprana de errores y la visualización inmediata de los cambios

realizados. Su utilización ha resultado especialmente útil durante el desarrollo, ya que evita la necesidad de recargas manuales y gestiona de forma transparente las rutas relativas entre los distintos recursos del proyecto.

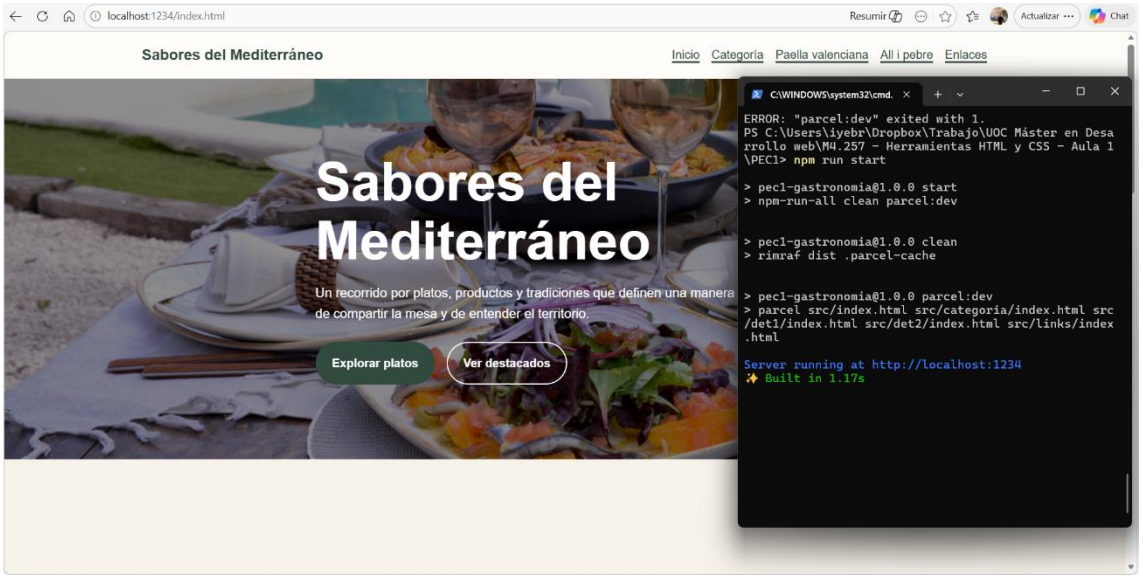

Figura 2. Entorno de desarrollo con Parcel*. Ejecución del proyecto en local mediante servidor de desarrollo con recarga automática y resolución dinámica de recursos.*

3. Entorno de desarrollo y entorno de producción

Durante el desarrollo se ha trabajado en un entorno local gestionado por Parcel, accesible mediante localhost. Este entorno proporciona:

- recarga automática del navegador
- detección inmediata de errores
- resolución dinámica de dependencias

Este flujo de trabajo facilita la iteración rápida, especialmente en fases de ajuste de estilos y depuración.

Por otro lado, el entorno de producción corresponde a la versión final del proyecto, preparada para su despliegue en un servidor.

Para garantizar su correcto funcionamiento en este entorno se han tenido en cuenta:

- uso de rutas relativas robustas
- estructura de archivos clara
- independencia respecto al entorno de desarrollo

La principal diferencia entre ambos entornos radica en que el entorno de desarrollo es dinámico y tolerante a errores, mientras que el de producción requiere una resolución explícita y correcta de todos los recursos.

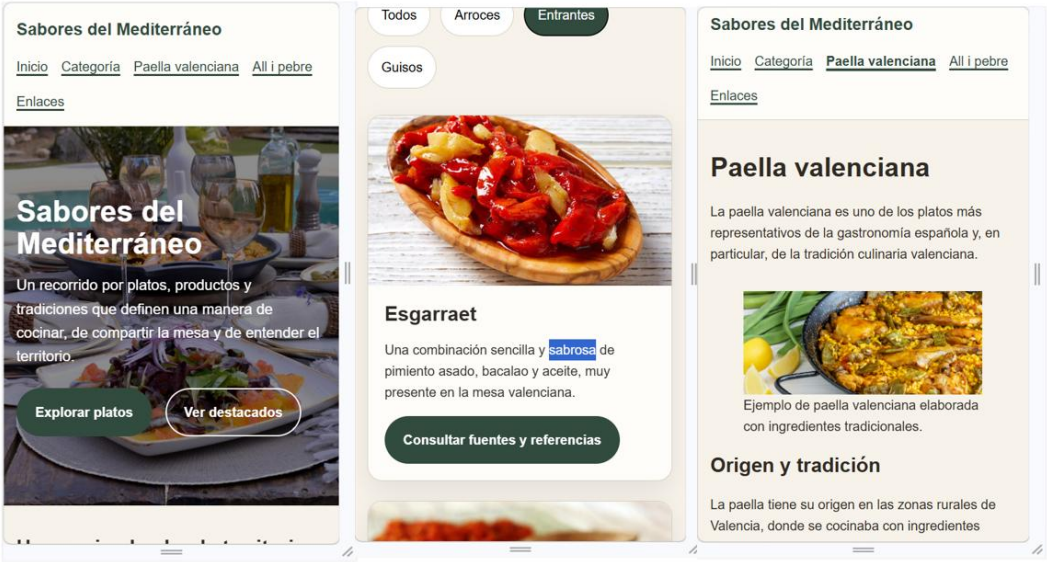

***Figura 3. Diseño responsivo**. Adaptación de la interfaz a dispositivos móviles mediante el uso de media queries y diseño flexible*

4. Desarrollo de la interfaz

La interfaz del proyecto se ha diseñado siguiendo un enfoque visual y estructurado, orientado a facilitar la navegación y la comprensión del contenido por parte del usuario. Se ha optado por una organización basada en bloques claramente diferenciados, manteniendo una coherencia estética entre las distintas páginas.

En particular, la página de categoría presenta los distintos platos mediante un sistema de tarjetas, cada una de ellas compuesta por una imagen representativa, un título y un enlace a su correspondiente página de detalle. Este diseño permite una exploración rápida del contenido y mejora la experiencia de usuario al ofrecer una visión global de las opciones disponibles.

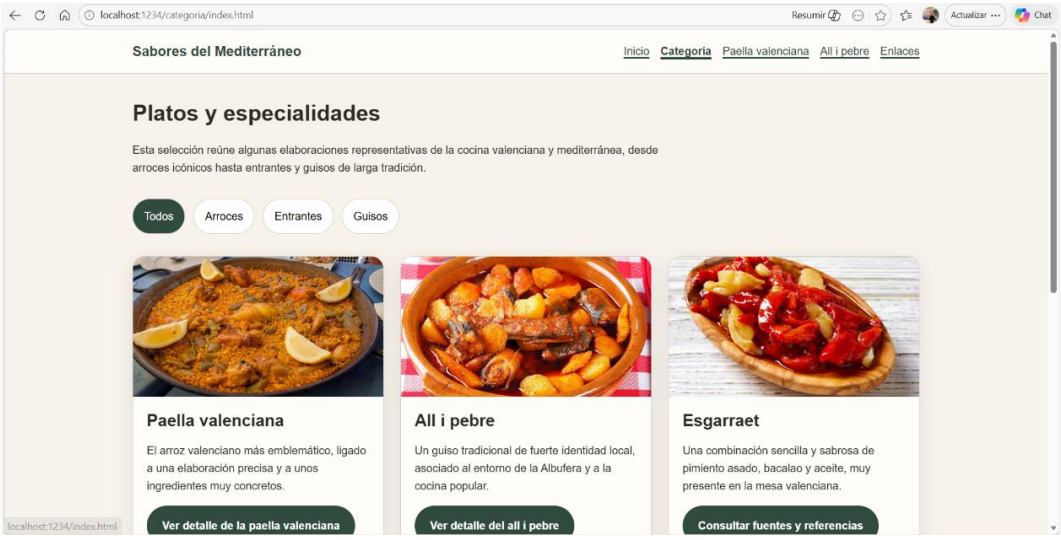

***Figura 4. Página de categoría.** Visualización de contenido organizado en tarjetas y sistema de filtrado dinámico implementado mediante JavaScript.*

Las páginas de detalle amplían la información de cada plato mediante una estructura más rica que combina distintos tipos de contenido. En ellas se integran imágenes destacadas, descripciones textuales, galerías y elementos multimedia, configurando así una presentación más completa y contextualizada.

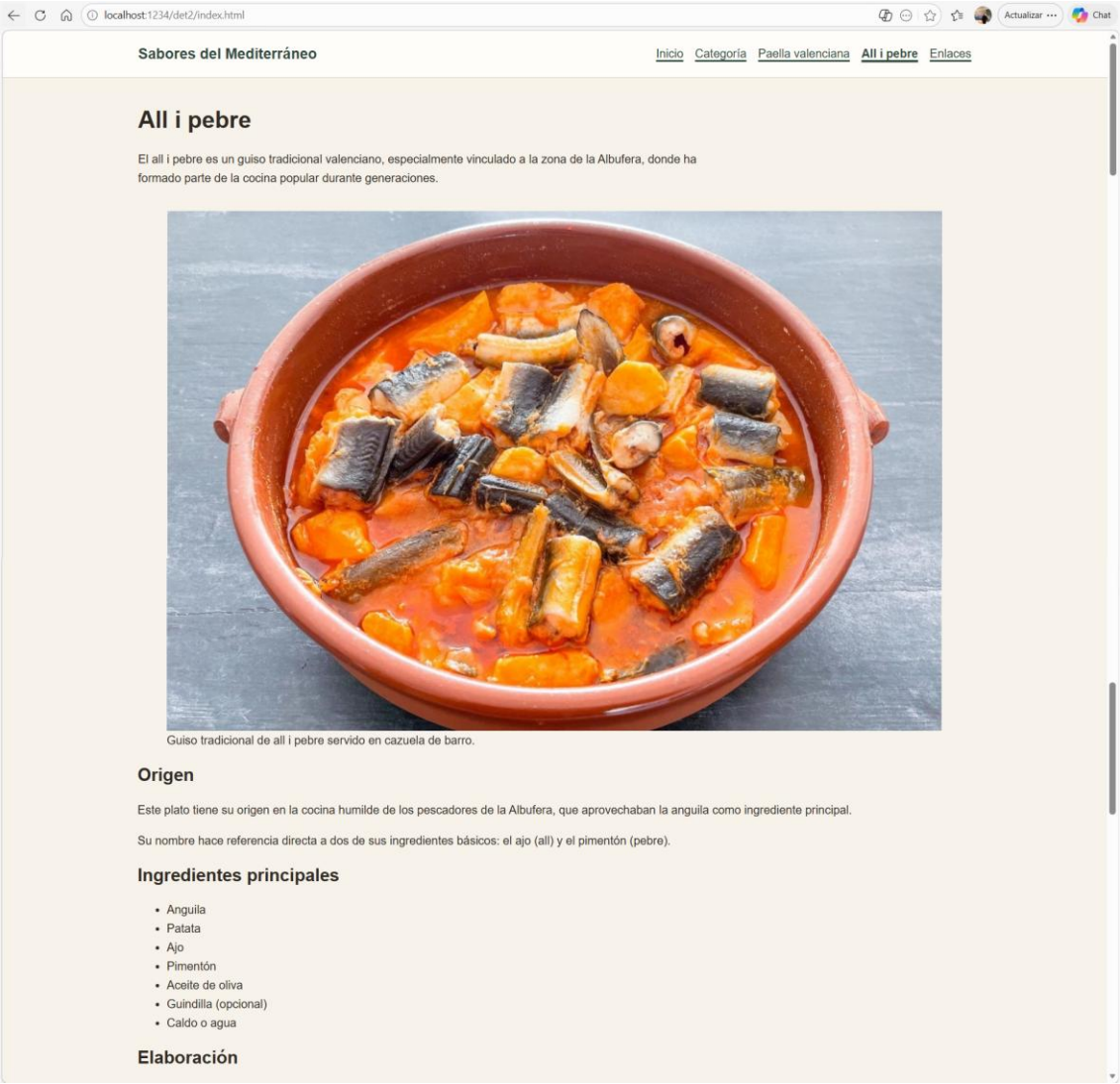

Figura 5. Página de detalle. *Ejemplo de estructura semántica con imagen destacada, contenido textual, galería de imágenes y vídeo incrustado.*

5. Compatibilidad con navegadores antiguos

El proyecto ha sido desarrollado teniendo en cuenta la compatibilidad con navegadores que no soportan completamente las características más modernas del lenguaje.

Para ello se han aplicado las siguientes estrategias:

- uso de propiedades CSS ampliamente soportadas
- evitación de funcionalidades experimentales
- dependencia limitada de JavaScript para funcionalidades críticas

Además, se ha utilizado una configuración de navegadores mediante browserslist, lo que permite a Parcel adaptar la salida del código en función de los navegadores objetivo.

Es importante destacar que la compatibilidad se ha entendido como funcionalidad básica garantizada, no como igualdad visual total entre navegadores.

Además, se ha tenido en cuenta el principio de degradación progresiva, de modo que el sitio mantiene su funcionalidad esencial incluso en navegadores con menor soporte de características modernas. Esto implica que, aunque ciertos efectos visuales o comportamientos avanzados puedan no estar disponibles en todos los entornos, el contenido sigue siendo accesible y navegable.

También se ha considerado la posible variabilidad en la interpretación de estilos entre navegadores, especialmente en aspectos como el renderizado de tipografías o el comportamiento de determinados elementos de maquetación. Por este motivo, se han utilizado soluciones contrastadas y se ha evitado depender de implementaciones específicas de un único navegador.

6. **Uso de preprocesadores y postprocesadores**

Aunque no se ha utilizado un preprocesador CSS como Sass, sí se ha trabajado con herramientas de procesamiento a través de Parcel, que actúa como postprocesador.

Parcel permite:

- importar CSS desde JavaScript
- gestionar automáticamente dependencias
- optimizar la carga de recursos

Ejemplo:

```
import "aos/dist/aos.css";
```

Este enfoque permite centralizar la gestión de estilos y dependencias.

Organización del CSS

Se ha optado por separar los estilos en dos archivos:

- estilos.css → estilos principales
- print.css → estilos específicos para impresión

Esta decisión responde a la necesidad de adaptar la presentación del contenido a distintos contextos de uso, cumpliendo uno de los requisitos de la práctica.

7. **Dependencias externas**

Se ha incorporado la librería AOS (Animate On Scroll) como dependencia externa, con el objetivo de enriquecer la experiencia visual mediante animaciones.

La integración se ha realizado mediante:

```
import AOS from "aos";
import "aos/dist/aos.css";

AOS.init();
```

Y su uso en HTML:

```
<div data-aos="fade-up">
```

Justificación

La elección de esta librería se basa en:

- su bajo impacto en la estructura del proyecto
- facilidad de integración
- mejora de la percepción visual sin comprometer la accesibilidad

Se trata de una dependencia no crítica, lo que significa que el sitio sigue siendo funcional incluso si la librería no se carga.

8. **Semántica y accesibilidad**

El uso de HTML semántico ha sido uno de los pilares del proyecto.

Se han empleado etiquetas adecuadas para cada tipo de contenido:

- header, nav, main, section, footer
- figure y figcaption
- listas (ul, ol)

Ejemplo:

```
<figure>
  <img src="..." alt="Paella valenciana tradicional">
  <figcaption>Ejemplo de paella valenciana</figcaption>
</figure>
```

Accesibilidad

Se han aplicado diversas medidas:

- uso sistemático de atributos alt en imágenes
- implementación de un enlace de salto al contenido principal
- uso de aria-label y aria-pressed en elementos interactivos
- inclusión de atributo title en iframes

Estas decisiones permiten mejorar la experiencia de usuarios con tecnologías de asistencia y cumplen con las recomendaciones básicas de accesibilidad web.

9. Uso de Git y GitHub

El proyecto ha sido gestionado mediante Git como sistema de control de versiones.

Se han realizado commits progresivos, agrupando cambios relacionados para mantener claridad en el historial.

El uso de Git permite:

- controlar la evolución del proyecto
- revertir cambios en caso de error
- mantener una estructura de trabajo organizada

El repositorio ha sido preparado para su publicación en GitHub, lo que facilita tanto la evaluación como el despliegue.

Durante el desarrollo, el uso de Git ha permitido trabajar de forma iterativa, registrando los distintos avances del proyecto mediante commits descriptivos. Esta práctica facilita no solo el seguimiento de los cambios realizados, sino también la identificación de posibles errores introducidos en fases concretas del desarrollo.

Asimismo, el control de versiones ha resultado útil para mantener una organización clara del trabajo, especialmente al abordar distintas partes del proyecto de manera progresiva. La posibilidad de revisar versiones anteriores ha aportado seguridad durante el desarrollo, evitando la pérdida de trabajo y permitiendo tomar decisiones con mayor control.

10. Publicación en internet

La publicación del sitio se ha planteado mediante servicios de hosting estático como Netlify.

El proceso consiste en:

1. subir el repositorio a GitHub
2. conectar el repositorio con Netlify
3. desplegar el proyecto automáticamente
4. obtener una URL pública

Este tipo de servicios permite publicar sitios web de forma sencilla, sin necesidad de configurar servidores manualmente.

11. Conclusiones

El desarrollo de esta práctica ha permitido aplicar de forma práctica los conceptos fundamentales de HTML y CSS, así como introducir herramientas modernas de desarrollo.

Más allá de la implementación técnica, se ha prestado especial atención a:

- la coherencia estructural del sitio
- la claridad del código
- la accesibilidad
- la mantenibilidad

El resultado es un proyecto funcional, estructurado y alineado con las buenas prácticas actuales del desarrollo web.